

Project Exposé: Improving Mediapipe Accuracy for Hand Control in Unreal Engine

Research Lead: Prof. Dr. Daniel Roth **Type:** Confidential Project Proposal **Type:** Bachelor Thesis
Author: Julian **Date:** 25.09.2023

Abstract

Hand tracking has become an essential component of natural interaction in gaming and virtual environments. While virtual reality (VR) devices provide accurate tracking, their high cost limits accessibility for many users. MediaPipe offers a low-cost alternative through a standard camera, making it attractive for immersive applications. However, directly feeding raw MediaPipe data into Unreal Engine often results in non-optimal visualization, such as jitter, noise, and mismatched scaling between the tracked landmarks and humanoid models.

This thesis investigates methods to improve the accuracy and realism of MediaPipe-based hand tracking in Unreal Engine. The approach involves finding a suitable filtering techniques (e.g., Kalman, One-Euro, and smoothing filters) and scaling strategies to reduce jitter and ensure consistency with humanoid skeletons. The expected outcome is a robust hand tracking pipeline that produces stable and realistic hand motion in Unreal Engine.

I. MOTIVATION

Games can offer highly immersive experiences by using Virtual Reality, but not everyone can access VR devices due to their high cost. MediaPipe, a hand-tracking framework developed by Google that requires only a standard camera, offers an affordable and accessible alternative. This thesis serves as a basis for MediaPipe-enabled game development for future applications, offering an alternative approach for immersive experiences without relying on VR .

MediaPipe can also be applied to therapy-focused games, where users perform specific tasks, such as moving their fingertips to target positions. Having accurate hand-tracking will be useful for rehabilitation exercises and may help improve motor skills through an enjoyable game interface.

II. PROBLEM

Without data processing, feeding raw MediaPipe data directly into Unreal Engine often results in a non-optimal hand visualization, e.g. fingers occasionally deviating from their intended positions. Causes of this issue include noise in the data (which causes jitter) as well as mismatched scaling between the MediaPipe and the humanoid model. Therefore, identifying suitable scaling and filtering methods to achieve realistic motion is a challenge.

III. RELATED WORK

Hand tracking and gesture recognition have been widely explored in recent years, with Mediapipe serving as one of the most commonly adopted frameworks. A number of works have focused on improving gesture recognition accuracy through filtering techniques. For instance, Zhang et al. [1] applied smoothing and Kalman filtering to MediaPipe outputs in order to stabilize landmarks for gesture recognition. Similarly, Li et al. [2] presented a comparable application but without detailing the filtering methods used. Other research has extended MediaPipe for more immersive experiences, such as the work of Kovács et al. [3], who introduced correction coefficients to enhance motion realism in interactive scenarios. Beyond MediaPipe, some studies

have explored the use of multiple sensors combined with Extended Kalman Filters [4], highlighting the general importance of noise reduction and motion stabilization in hand-tracking systems.

MediaPipe has also been applied in real-time monitoring contexts. For example, Chen et al. [5] combined MediaPipe and OpenCV to implement real-time gesture recognition and exercise monitoring, demonstrating its potential in healthcare and rehabilitation. Similarly, Srecharan [6] experimented with MediaPipe-based real-time hand rigging, although the focus remained on gesture recognition rather than full-hand visualization accuracy.

A smaller set of works have attempted to integrate MediaPipe directly into game engines. The Mediapipe4U plugin [7] enables Unreal Engine users to leverage AI-driven motion capture but does not provide systematic details on filtering or scaling. Likewise, UnityHandTrackingWithMediapipe [8] demonstrates a working integration in Unity, but it lacks documentation on pipeline design, filtering, and inverse kinematics retargeting.

In summary, most published works concentrate on gesture recognition and do not explicitly describe a proper setup for real-time hand tracking in Unreal Engine with appropriate filtering, scaling, and retargeting strategies. This leaves a gap in the literature for research focused on stabilizing and optimizing MediaPipe data for realistic visualization in advanced 3D environments such as Unreal Engine, which this thesis aims to address.

IV. APPROACH

The proposed approach will focus on improving hand gesture accuracy and realism in Unreal Engine using MediaPipe data:

- 1) Data Filtering and Scaling : Test various filtering algorithms (e.g., Kalman , moving average) and scaling strategies to reduce noise and improve motion fidelity.
- 2) Control Rig Optimization : Configure forward and inverse kinematics in Unreal Engine to map MediaPipe joint positions to a humanoid hand accurately. In Unreal Engine, the control rig allows developers to manipulate bones of the model, implement FK and IK, as well as create a custom control.
- 3) Data Simplification : Evaluate the use of a subset of key points, such as fingertips, to simplify control while maintaining realistic hand gestures instead of using all joints data.
- 4) Validation: Compare the generated hand gestures with intended motions to determine the most effective combination of filtering, scaling, and IK configuration. To have a quantitative analysis , we can compare a sample hand gesture with the actual hand gesture performed by the user and then calculate their similarity.

V. GOALS/OBJECTIVE

The goal is to identify the most optimal setup for translating MediaPipe hand-tracking data into natural, responsive hand movements in Unreal Engine. The optimal solution includes finding an effective filtering and scaling, as well as configuring forward and inverse kinematics for a humanoid model. The thesis can serve as a foundation for a reliable pipeline to use MediaPipe hand-tracking data in Unreal Engine.

VI. SCHEDULE/TIME PLAN

Bachelor thesis in games engineering field lasts maximum 4 months.

- 1) Month 1: Review literature on filtering, scaling, and IK in hand tracking. Finalize precise goal of this thesis and setup the project in Unreal Engine including all required resources including identify a way to feed Mediapipe data to Unreal Engine.

- 2) Month 2: Setup control rig and kinematics in unreal engine, explore several IK solvers, visualize how raw data Mediapipe would look like, find the best scaling method.
- 3) Month 3: evaluate several filtering methods, and find a suitable one, compare fingertip-driven to joint-driven approaches, combine filtering + scaling + IK into one pipeline.
- 4) Month 4: Evaluate accuracy and realism (qualitative comparison, maybe small user test), finalize paper and prepare paper defense.

VII. RELATED EXPERIENCE

- 1) Experienced in setting up Mediapipe in Python, extracting positional data, and feeding it into Unreal Engine.
- 2) Knowledge of Unreal Engine's Control Rig system, including setup FK and IK of a humanoid model.
- 3) Proficient in integrating pybind11 into Unreal Engine to execute Python scripts, allowing use of scientific libraries like NumPy for real-time data filtering.
- 4) Familiar with retargeting in Unreal Engine to apply hand motion to more sophisticated models, such as Metahuman.

VIII. WHY UNREAL ENGINE

Unreal Engine offers several advantages for this project:

- 1) Provides a humanoid mannequin and a fully featured Control Rig for custom forward and inverse kinematics setups.
- 2) Offers multiple IK solvers for realistic motion, for example FABRIK (Forward and Backward Reaching Inverse Kinematics), Two-Bone IK, etc
- 3) Supports integration with Python via pybind11, enabling advanced numerical processing and filtering.
- 4) Facilitates retargeting, allowing adaptation of hand motion to different models (e.g., Metahuman), which enhances realism.

REFERENCES

- [1] Yuecheng Fan. 2024. The Gesture Recognition Improvement of Mediapipe Model Based on Historical Trajectory Assist Tracking, Kalman Filtering and Smooth Filtering. In Proceedings of the 2024 7th International Conference on Computer Information Science and Artificial Intelligence (CISAI '24). Association for Computing Machinery, New York, NY, USA, 641–647. <https://doi.org/10.1145/3703187.3703295>
- [2] Q. Zhou and B. G. Lee, "Optical motion capture application based on Unity3D and MediaPipe," 2024 3rd International Conference on Cloud Computing, Big Data Application and Software Engineering (CBASE), Hangzhou, China, 2024, pp. 618–623, doi: 10.1109/CBASE64041.2024.10824452.
- [3] Zhao, S., Ozaki, T. (2025). Enhancing VR Immersion Through Avatar Scaling and Sensor Fusion with Mediapipe. In: Chen, J.Y.C., Fragomeni, G. (eds) Virtual, Augmented and Mixed Reality. HCII 2025. Lecture Notes in Computer Science, vol 15790. Springer, Cham. https://doi.org/10.1007/978-3-031-93715-6_5
- [4] Lei, Y., Deng, Y., Dong, L., Li, X., Li, X., & Su, Z. (2023). A Novel Sensor Fusion Approach for Precise Hand Tracking in Virtual Reality-Based Human—Computer Interaction. *Biomimetics*, 8(3), 326. <https://doi.org/10.3390/biomimetics8030326>
- [5] J. Vijaya, A. Singh, K. Singh and A. Kumar, "AI-Enabled Real-Time Exercise Monitoring with MediaPipe and OpenCV," 2024 15th International Conference on Computing Communication and Networking Technologies (ICCCNT), Kamand, India, 2024, pp. 1–5, doi: 10.1109/ICCCNT61001.2024.10724252.
- [6] Srecharan, "Mediapipe-Based Real-Time Hand Rigging," 2023. [Online]. Available: https://srecharan.github.io/projects/3_project/.
- [7] Endink, "Mediapipe4U Plugin for Unreal Engine," GitHub Repository, 2023. [Online]. Available: <https://github.com/endink/Mediapipe4u-plugin>.
- [8] TesseractZero, "UnityHandTrackingWithMediapipe," GitHub Repository, 2022. [Online]. Available: <https://github.com/TesseractZero/UnityHandTrackingWithMediapipe>.

This proposal is confidential. Do not forward, copy, or distribute this text without explicit consent.

Correspondence: daniel.roth@tum.de.